EASy68K Web

TRAP #15 Reference

Antoni Burguera
Universitat de les Illes Balears

September 8, 2026

About this reference

`DO.B` selects the task before every `TRAP #15`. This is the real TRAP #15 dispatch logic from the original EASy68K source – every task listed here does exactly what the table says, including the ones this web port implements in the browser rather than through Windows GDI. Text and graphics draw to the **same screen**, exactly like real EASy68K (text is drawn as glyphs onto the graphics canvas by row/column). Only serial (40–43) and network (100+) are unimplemented, by explicit choice: a browser tab cannot open a serial port or an arbitrary TCP socket, so those tasks log a one-time explanation and report a definite failure code instead of hanging or crashing.
Every register is given with its explicit size (`.B` / `.W` / `.L`). Where a task packs several values into one register, that is spelled out explicitly (e.g. `D1.L packed: bits 16--31 = row, bits 0--15 = col`). This same content, always kept in sync with the running application, is also available from inside the app itself: click **Help** (the question-mark icon), then the **TRAP #15** tab. For background on TRAP #15 itself, or any real-EASy68K behavior this reference doesn't spell out, see EASy68K's own official documentation at http://www.easy68k.com/.

Contents

TEXT / SYSTEM

Task	Parameters	Action
0	D1.W = length	Display string at (A1), D1.W bytes, then CR/LF
1	D1.W = length	Display string at (A1), D1.W bytes, no CR/LF
2	D1.L (out) = length	Read a string into the buffer at (A1); returns length read
3	D1.L = value	Display D1.L as a signed decimal number
4	D1.L (out)	Read a number (prompted) into D1.L
5	D1.B (out)	Read a single character (prompted) into D1.B
6	D1.B = char	Display a single character
7	D1.B (out)	Set D1.B to 1 if keyboard input is pending, else 0
8	D1.L (out)	Return time of day: hundredths of a second since midnight
9	—	Terminate the program (SIMHALT)
10	A1 = text ptr	Print text at (A1) — unavailable in a browser, logged only

Task	Parameters	Action
11	D1.W: $FF00=clear, $00FF=get pos (out), else low/high byte = col/row	Position the text cursor, clear the screen, or read the cursor position
12	D1.B = 0 (off) / non-zero (on)	Keyboard echo on/off
13	A1 = NUL-terminated string	Display NUL-terminated string at (A1), then CR/LF
14	A1 = NUL-terminated string	Display NUL-terminated string at (A1), no CR/LF
15	D1.L = value, D2.B = radix	Display D1 converted to the given radix (e.g. 16 for hex)
16	D1.B = 0 (off) / non-zero (on)	Input prompt on/off
17	A1 = NUL-terminated string, D1.L = value	Display NUL-terminated string at (A1), no CR/LF, then display D1.L as a decimal number, no CR/LF
18	A1 = NUL-terminated string, D1.L (out)	Display NUL-terminated string at (A1), no CR/LF, then read a number (prompted) into D1.L
19	D1.L = 0 for last key codes, or up to 4 packed key codes to test; D1.L (out) = result	If D1.L in is 0, return the last key-up code (bits 16-23) and key-down code (bits 0-7); otherwise test up to four packed key codes and return $FF in each byte whose key is currently held down
20	D1.L = value, D2.B = field width	Display D1.L as a zero/space-padded decimal number
21	D1.L = BGR color, D2.L packed: bits 0-15 (D2.W) = style bits (bold=1, italic=2, underline=4, strike=8), bits 16-23 = size in points, bits 24-31 = font id (unused)	Set the text font color, style, and size - the bitmap font only supports integer scale steps, so size is rounded to the nearest multiple of the native 8pt size. Load D2 as a full .L - the size lives above bit 15, so a plain MOVE.W into D2 would leave it untouched
22	D1.L packed (in): bits 16-31 = row, bits 0-15 = col; D1.B (out) = char	Return the character currently at the given row/col
23	D1.L = hundredths of a second	Delay - a real wall-clock pause while Run is active
24	D1.B	Enable/disable shortcut keys - no effect (no desktop-style shortcuts to suppress in a web port)
25	D1.L packed: bits 16-31 = row, bits 0-15 = col; D2.L packed: bits 16-31 = height, bits 0-15 = width; D3.W = direction (0=up, 1=down)	Scroll a text rectangle up or down
30	—	Clear the instruction cycle counter
31	D1.L (out)	Return the instruction cycle counter
32.0	D1.B(in) = 0	Display the hardware window (7-segment, LEDs, switches)
32.1	D1.B(in) = 1; D1.L(out) = address	Return the memory address of the 7-segment display
32.2	D1.B(in) = 2; D1.L(out) = address	Return the memory address of the LEDs
32.3	D1.B(in) = 3; D1.L(out) = address	Return the memory address of the toggle switches
32.4	D1.B(in) = 4; D1.L(out) = version	Return the simulator version number
32.5	D1.B(in) = 5	Enable exception processing
32.6	D1.B(in) = 6; D2.B = mask (bit 7 = enable, bits 0-6 = IRQ level), D3.L = period in ms	Set up (or cancel) auto-repeating IRQ generation for a hardware level - same underlying state as the Hardware window's own "Auto every" checkbox/interval for that level

Task	Parameters	Action
32.7	D1.B(in) = 7; D1.L(out) = address	Return the memory address of the push-button switches
33	D1.L=1 windowed, D1.L=2 fullscreen, D1.L=W«16|H set size, D1.L=0 get size (out, packed the same way)	Get/set the output window mode and size (fullscreen via the real browser Fullscreen API - Esc, or the window's own maximize/restore button, to exit)

FILE I/O

Task	Parameters	Action
50	D0.W(out) = result	Close all open files
51	A1 = filename, D1.L(out) = handle, D0.W(out) = result	Open an existing file for read/update (falls back to read-only if it isn't writable)
52	A1 = filename, D1.L(out) = handle, D0.W(out) = result	Create a new file (or truncate an existing one) for read/write
53	D1.L=handle, A1=buffer, D2.L(in)=bytes to read, D2.L(out)=bytes actually read, D0.W(out)=result	Read from a file
54	D1.L=handle, A1=buffer, D2.L=bytes, D0.W(out)=result	Write to a file
55	D1.L=handle, D2.L=absolute byte offset from the start of the file, D0.W(out)=result	Position (seek) within a file
56	D1.L = handle, D0.W(out) = result	Close a file
57	A1 = filename, D0.W(out) = result	Delete a file
58	D1.L(in)=dialog mode (0=Open, else Save - label text only, behavior is the same either way), D1.L(out)=1 if a file was picked/0 if canceled, A1=title string ptr (or 0), A2=extension filter string ptr (or 0, not used to filter here), A3=path buffer (in: initial value; out: chosen filename)	Show a file picker - implemented as a prompt() listing the files already in the virtual file system, since a browser can't show a native OS file dialog
59	A1 = filename, D0.W(out) = result	Check whether a file exists and is writable, without opening it (D1 has no effect in this build - the real task's mode/sub-operation switch was never implemented on either the original or this port's file-op routine, which unconditionally just probes the file)

Every file task above returns its result code in D0.W: 0 = success, 1 = end-of-file (task 53 only), 2 = error, 3 = opened read-only (tasks 51/59 only, when the file exists but couldn't be opened for writing).

MOUSE / KEYBOARD

Task	Parameters	Action
60	D1.W packed: bits 8-15 = IRQ level 0-7, bits 0-2 = arm button-down/button-up/move	Arm the mouse button-down/up/move IRQ(s) at the given level (a level >7 is ignored) - click the graphics screen to give it focus first

Task	Parameters	Action
61	D1.W(out)=X, D2.W(out)=Y, D3.B(out)=buttons	Read the current mouse position and button state
62	D1.W packed: bits 8-15 = IRQ level 0-7, bits 0-1 = arm key-down/key-up	Arm the keyboard key-down/up IRQ(s) at the given level (a level >7 is ignored) - click the graphics screen to give it focus first

SOUND

Task	Parameters	Action
70	A1 = WAV filename, D0.W(out) = result	Play a WAV file directly, one-shot (from the uploaded virtual filesystem, via Web Audio)
71	A1 = WAV filename, D1.B(in) = slot to load it into	Load a WAV into a player slot you choose, for fast replay later via task 72 (no result code is returned by this build)
72	D1.B(in) = slot, D0.W(out) = result	Play a previously-loaded WAV from the given slot
73	A1 = WAV filename, D0.W(out) = result	Play a WAV file directly, one-shot ("DirectX" player variant)
74	A1 = WAV filename, D1.B(in) = slot to load it into, D0.W(out) = result	Load a WAV into a "DirectX" player slot you choose
75	D1.B(in) = slot, D0.W(out) = result	Play a previously-loaded WAV from the given "DirectX" slot
76	D1.B = slot, D2.L = command, D0.W(out) = result	Control the standard player for a loaded slot (stop/pause/volume, depending on the D2 command value)
77	D1.B = slot, D2.L = command, D0.W(out) = result	Control the "DirectX" player for a loaded slot (stop/pause/volume, depending on the D2 command value)

GRAPHICS

Task	Parameters	Action
80	D1.L = BGR color	Set the pen (line/outline) color
81	D1.L = BGR color	Set the fill color
82	D1.W = X, D2.W = Y	Draw a single pixel
83	D1.W=X, D2.W=Y, D1.L(out)=color	Read the color of a pixel
84	D1.W=X1, D2.W=Y1, D3.W=X2, D4.W=Y2	Draw a line between two points
85	D1.W = X, D2.W = Y	Draw a line from the current pen position to (X,Y)
86	D1.W = X, D2.W = Y	Move the pen to (X,Y) without drawing
87	D1.W=X1, D2.W=Y1, D3.W=X2, D4.W=Y2	Draw a filled rectangle
88	D1.W=X1, D2.W=Y1, D3.W=X2, D4.W=Y2	Draw a filled ellipse (bounding box)
89	D1.W = X, D2.W = Y	Flood-fill starting at (X,Y)
90	D1.W=X1, D2.W=Y1, D3.W=X2, D4.W=Y2	Draw an unfilled (outline only) rectangle
91	D1.W=X1, D2.W=Y1, D3.W=X2, D4.W=Y2	Draw an unfilled (outline only) ellipse

Task	Parameters	Action
92	D1.L = mode (16=single-buffer, 17=double-buffer, others=pen mode)	Set the drawing/pen mode
93	D1.L = width	Set the pen width
94	—	Repaint - explicit full flip from the back buffer to the visible screen (only meaningful in double-buffer mode)
95	A1=NUL-terminated string, D1.W=X, D2.W=Y	Draw a string on the graphics canvas at (X,Y)
96	D1.W (out) = X, D2.W (out) = Y	Return the current pen position

NETWORK / SERIAL (unimplemented, by choice)

Serial (40-43) and networking (100-107) log a one-time explanation and report a definite failure code, so a program checking the result can detect it and continue gracefully.

WEB PORT EXTENSIONS (200-255 reserved)

Tasks 200-255 are reserved exclusively for this web port's own debugging/convenience additions, so they can never collide with a genuine task number the real EASy68K might add.

Tasks 200/201/203/206/213/214/215/217/220 are the CONSOLE_LOG family: exact replicas of the real text-printing tasks 0/1/3/6/13/14/15/17/20 above (same task number + 200), except they write to the Log panel (in the same distinctive amber color) instead of the screen/output window. The real tasks 0/1/3/6/13/14/15/17/20 themselves are completely unaffected.

Task	Parameters	Action
200	A1 = string, D1.W = length	CONSOLE_LOG - logs D1.W characters at (A1) to the Log panel (replica of task 0).
201	A1 = string, D1.W = length	CONSOLE_LOG - logs D1.W characters at (A1) to the Log panel (replica of task 1).
203	D1.L = number	CONSOLE_LOG - logs the decimal value of D1.L to the Log panel (replica of task 3).
206	D1.B = character	CONSOLE_LOG - logs the single character in D1.B to the Log panel (replica of task 6).
213	A1 = NUL-terminated string	CONSOLE_LOG - logs the string at (A1) to the Log panel (replica of task 13).
214	A1 = NUL-terminated string	CONSOLE_LOG - logs the string at (A1) to the Log panel (replica of task 14).
215	D1.L = number, D2.B = radix (2-36)	CONSOLE_LOG - logs D1.L converted to the given radix to the Log panel (replica of task 15).
217	A1 = NUL-terminated string, D1.L = value	CONSOLE_LOG - logs the string at (A1) followed by D1.L as a decimal number, on one Log panel line (replica of task 17).
220	D1.L = number, D2.B = field width	CONSOLE_LOG - logs D1.L right-justified in a D2.B-wide field to the Log panel (replica of task 20).

This table covers every TRAP #15 task as implemented in this web port. For background on TRAP #15 itself, or any real-EASy68K behavior this table doesn't spell out, see EASy68K's own official documentation at easy68k.com.